



Happy LDOC!



Function-Writing Practice
& Algorithmic Complexity Review

Reminders

Re: Assignments:

- **EX09: Linked List Utility Functions** due tonight at 11:59pm
 - Late submission deadline: 4pm on April 30th (start of the final exam)
 - You can work with a partner (or on your own)!!

Re: Final Exam:

- **Hybrid Review Session** on Wednesday, April 29 @ 3:30-5:30pm in SN014 (Sitterson Hall, room 014)
- **4–7pm on Thursday, April 30**
 - Makeup final exam on Friday, May 1 in SN014
 - If you qualify for the makeup, please complete our [internal form](#) ASAP. You will get a confirmation email next week!

[Apply to be a TA](#) for COMP110 in Fall 2026! Deadline is May 3!

Function-writing practice

Write a function called **shape** that:

- Has one parameter, **input_data_table**, of type `dict[str, list[str]]`
- It should return a `dict[str, int]` where each key is a column name from **input_data_table** and its corresponding value is the number of elements in that column.

Example behavior:

```
>>> data = { "name": ["Alice", "Bob", "Carol"], "major": ["CS", "Math"] }
```

```
>>> shape(data)
```

```
{"name": 3, "major": 2}
```

Function-writing practice

```
def shape(input_data_table: dict[str, list[str]]) -> dict[str, int]:  
    result: dict[str, int] = {}  
  
    for column in input_data_table:  
        result[column] = len(input_data_table[column])  
  
    return result
```

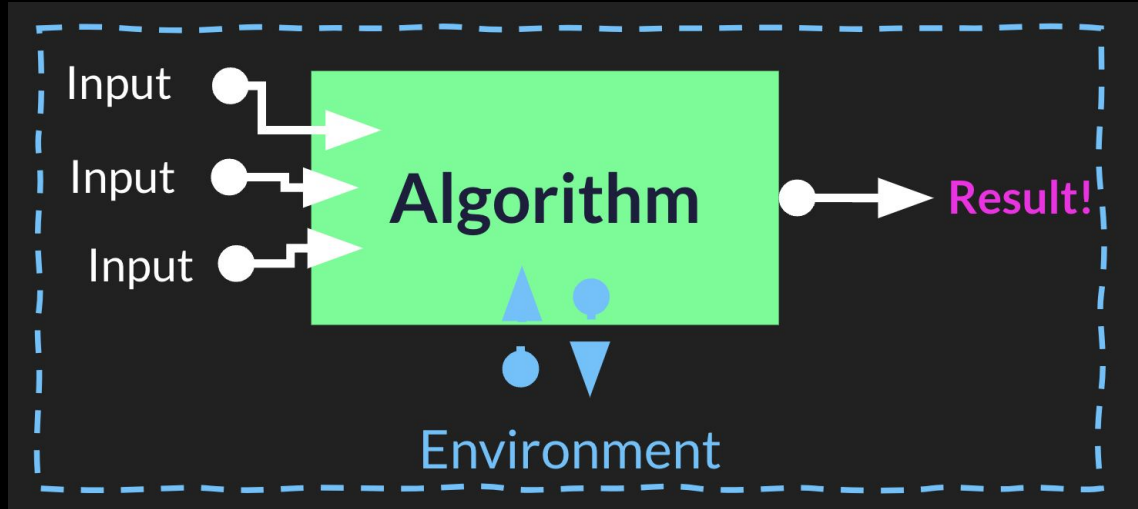
Review: Algorithms

Input is data given to an algorithm

An **algorithm** is a series of steps

An algorithm **returns** some **result**

An algorithm *may* be influenced by its **environment** and it *may* produce side-effects which influence its environment.



Running time: how long does an algorithm take to run?

- Empirical analysis: write the code and test how long it takes to run!
 - Weaknesses:
 - You have to write the code for the whole algorithm and run it to see how long it will take
 - Different computers with different specs will have different runtimes
- Rather than using empirical analysis, computer scientists commonly consider the **number of operations (steps)** an algorithm requires
 - 1 operation == 1 step

Runtime analysis: Best, average, and worst case

Best case (lower bound):

- Minimum number of operations (running time) required for the algorithm to execute

Average case:

- Average running time among several different inputs

Worst case (upper bound) ✨:

- The maximum running time given an input
 - How does the number of operations grow as an input grows?
- Important to understand how our algorithm will perform in the *worst* case
 - Prepare for the worst case. If an input ends up requiring fewer operations, great!!

Big O notation

The complexity hierarchy (fastest to slowest, asymptotically)

$O(1)$

Constant

$O(\log n)$

Logarithmic

$O(n)$

Linear

$O(n \log n)$

Linearithmic

$O(n^2)$

Quadratic

$O(2^n)$

Exponential

The complexity hierarchy (fastest to slowest, asymptotically)

$O(1)$

Constant

$O(\log n)$

Logarithmic

$O(n)$

Linear

$O(n \log n)$

Linearithmic

$O(n^2)$

Quadratic

$O(2^n)$

Exponential

Runtime analysis: $O(1)$ – Constant Time

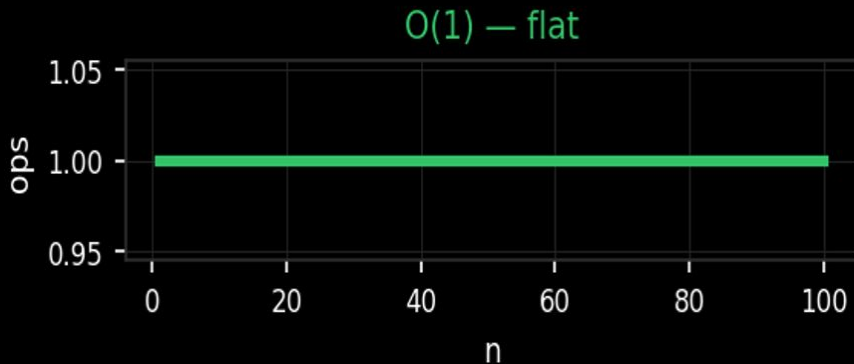
A constant-time operation takes the same number of steps, *regardless of the input size*.

Example:

Retrieving an element at index 0 of a list with 10 elements takes the same time as retrieving it from a list with 10,000,000 elements!

```
def get_first(items: list[int]) -> int:
    """Return the first item.  $O(1)$ ."""
    return items[0]    # ← one step!
```

```
def is_even(n: int) -> bool:
    """Is n even?  $O(1)$ ."""
    return n % 2 == 0  # ← one step!
```



Runtime analysis: $O(n)$ – Linear Time

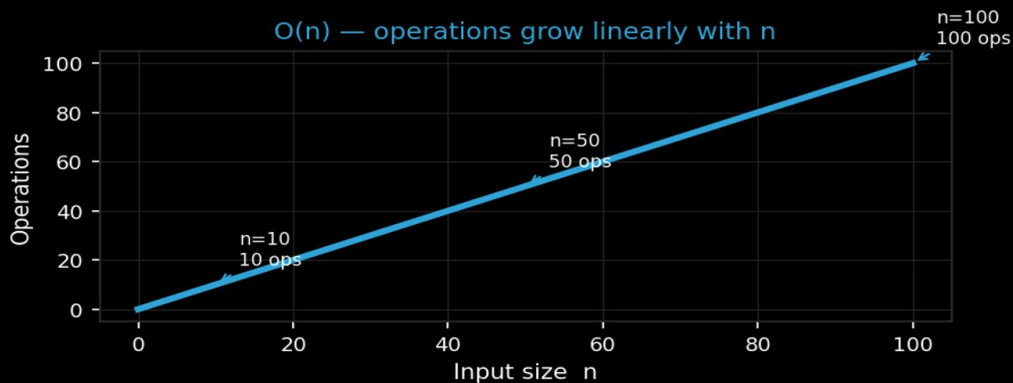
Runtime grows proportionally to the input size.

Example:

Printing the values in a list with 200 elements will take *twice as many operations* as printing the values in a list with 100 elements!

```
def print_all(items: list[str]) -> None:
    """Print every item.  $O(n)$ ."""
    for item in items:    # ← one step per item
        print(item)
```

```
def total(numbers: list[int]) -> int:
    """Add up all numbers in a list.  $O(n)$ ."""
    result = 0
    for n in numbers:
        result += n
    return result
```



Runtime analysis: $O(n^2)$ – Quadratic Time

Runtime grows quadratically with the size of the input.

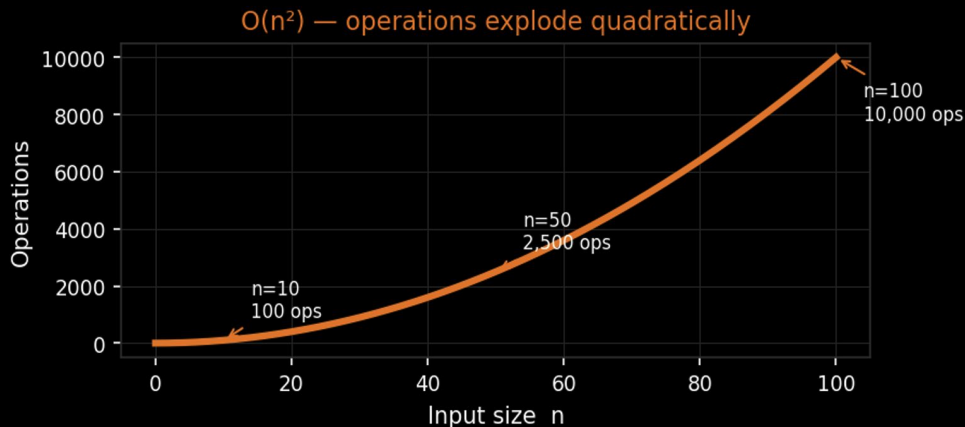
Example:

Nested lists! For every one of the n iterations of the outer loop, you do n iterations of the inner loop.

$2^2 \rightarrow 4$ operations

$600^2 \rightarrow 360,000$ operations

```
def print_all_pairs(lst: list[int]) -> None:
    """Print every pair of items.  $O(n^2)$ ."""
    for i in lst:           # ← outer loop
        for j in lst:       # ← inner loop
            print(f`{i}, {j}`) #  $n \times n = n^2$ 
```



Recall: comparing lists and sets

```
1 def intersection(a: list[str], b: list[str]) -> list[str]:
2     result: list[str] = []
3
4     idx_a: int = 0
5     while idx_a < len(a):
6         if a[idx_a] in b:
7             result.append(a[idx_a])
8             idx_a += 1
9
10    return result
```

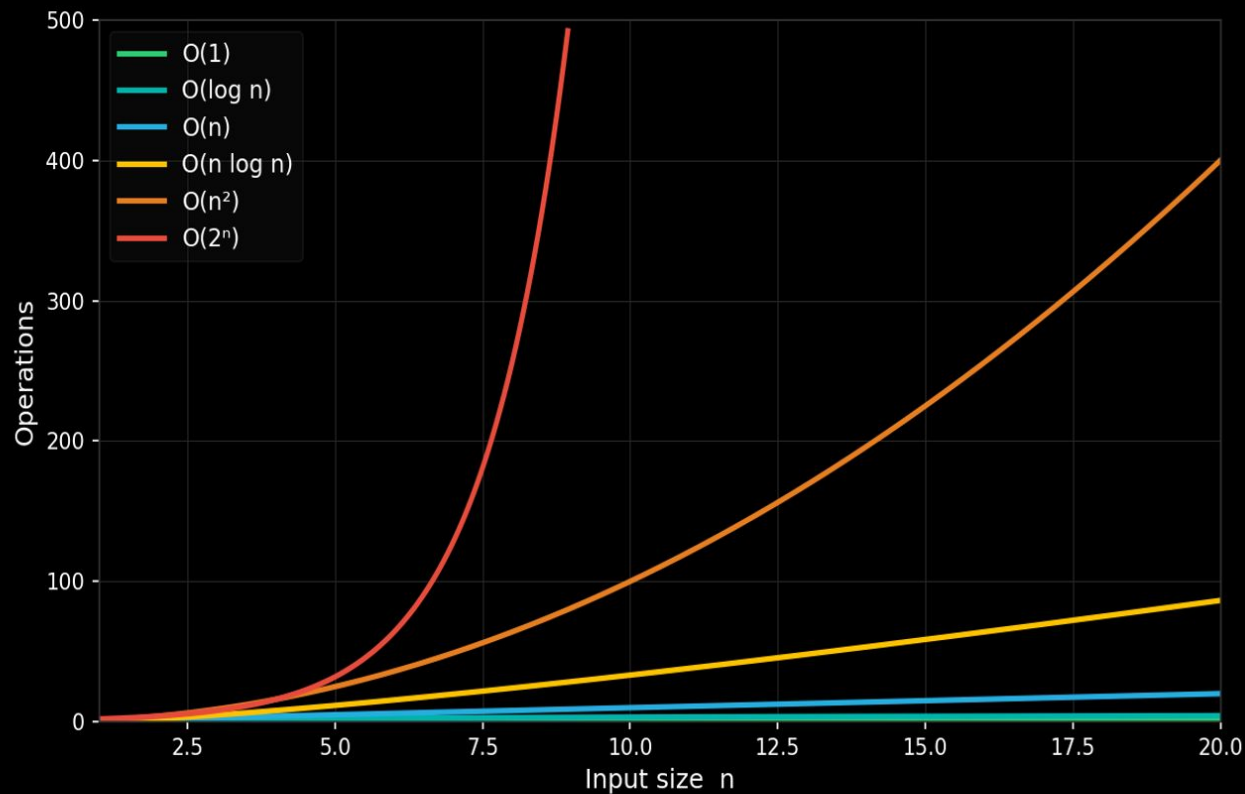
```
1 def intersection(a: list[str], b: set[str]) -> set[str]:
2     result: set[str] = set()
3
4     idx_a: int = 0
5     while idx_a < len(a):
6         if a[idx_a] in b:
7             result.add(a[idx_a])
8             idx_a += 1
9
10    return result
```

Suppose **a** and **b** each had 1,000,000 elements. The worst case difference here is approximately 1,000,000 operations, versus $1,000,000^2$ or 1 *trillion* (1,000,000,000,000) operations.

If your device can perform 100,000,000 operations per second, then...

A call to **a** will complete in 2.78 hours and **b** will complete in 1/100th of a second.

Growth rates, visualized



Thank you for a great semester!

, your COMP110 Team

P.S. Please complete your Course Evaluation;
it helps us improve the course!